GR22 Regulations

IV B.Tech I Semester

DEEP LEARNING LAB
(GR22A4081)

Department of Computer Science and Engineering
(Artificial Intelligence and Machine Learning)

GOKARAJU RANGARAJU
INSTITUTE OF ENGINEERING AND TECHNOLOGY
(Autonomous)

SYLLABUS

GOKARAJU RANGARAJU INSTITUTE OF ENGINEERING AND TECHNOLOGY

DEEP LEARNING LAB

Course Code: GR22A4081 **L/T/P/C: 0/0/4/2**

IV Year I Semester

COURSE OUTCOMES:

1. Learn the ANN models and Supervised Learning Models
2. Learn the Fundamental Principles of Deep Learning.
3. Identify the Deep Learning Algorithms for Various Types of Learning Tasks in various domains.
4. Implement Deep Learning Algorithms and Solve Real-world problems.
5. Apply Generative Algorithms

LIST OF EXPERIMENTS:

TASK-1: Setting up the Spyder IDE Environment and Executing a Python Program .

TASK-2: Installing Keras, Tensorflow and Pytorch libraries and making use of them.

TASK-3: Implement Activation Functions in Neural Networks and analyse their usage.

TASK-4: Implement Perceptron training algorithm to classify flowers in IRIS dataset.

TASK-5: Build a three-layer Artificial Neural Network by implementing the Back propagation algorithm.

TASK-6: Applying the Convolution Neural Network on computer vision problems

TASK-7: Image classification on MNIST dataset (CNN model with Fully connected layer)

TASK-8: Applying the Deep Learning Models in the field of Natural Language Processing.

TASK-9: Train a sentiment analysis model on IMDB dataset, use RNN layers with LSTM/GRU notes.

TASK-10: Applying the Autoencoder algorithms for encoding the real-world data.

TASK-11: Applying Generative Adversial Networks for image generation and unsupervised tasks.

TASK-12: Design a Deep Learning framework for Object Detection. GANS.

TEXT BOOKS:

1. Deep Learning by Ian Goodfellow, Yoshua Bengio and Aaron Courville, MIT Press.
2. The Elements of Statistical Learning by T. Hastie, R. Tibshirani, and J. Friedman, Springer.
3. Probabilistic Graphical Models. Koller, and N. Friedman, MIT Press.

REFERENCES:

1. Bishop, C.M., Pattern Recognition and Machine Learning, Springer, 2006.
2. Yegnanarayana, B., Artificial Neural Networks PHI Learning Pvt. Ltd, 2009.
3. Golub, G.H., and Van Loan, C.F., Matrix Computations, JHU Press, 2013.
4. Satish Kumar, Neural Networks: A Classroom Approach, Tata McGraw Hill Education,2004.

INDEX

S.NO.	Name of the Task	Page No.
1	Setting up the Spyder IDE Environment and Executing a Python Program.	5-5
2	Installing Keras, Tensorflow and Pytorch libraries and making use of them.	6-8
3	Implement Activation Functions in Neural Networks and analyse their usage.	9-12
4	Implement Perceptron training algorithm to classify flowers in IRIS dataset.	13-16
5	Build a three-layer Artificial Neural Network by implementing the Back propagation algorithm.	17-20
6	Applying the Convolution Neural Network on computer vision problems.	21-22
7	Image classification on MNIST dataset (CNN model with Fully connected layer).	23-24
8	Applying the Deep Learning Models in the field of Natural Language Processing.	25-26
9	Train a sentiment analysis model on IMDB dataset, use RNN layers with LSTM/GRU notes.	27-27
10	Applying the Autoencoder algorithms for encoding the real-world data.	28-29
11	Applying Generative Adversarial Networks for image generation and unsupervised tasks.	30-32
12	Design a Deep Learning framework for Object Detection. GANS.	33-36

TASK-1: Setting up the Spyder IDE Environment and Executing a Python Program

AIM: Download and install the Spyder and Executing a Hello World Program

Step 1: Installing Spyder

Spyder is included in the Anaconda Distribution, which is the recommended way to install it. Follow these steps:

1. Download and Install Anaconda:
   - Visit Anaconda's official website and download the latest version for your operating system.
   - Run the installer and follow the installation steps.
2. Verify Installation:
   - Open Anaconda Navigator and check if Spyder is listed.
   - Alternatively, open a terminal (or Anaconda Prompt) and type:

     ```
     spyder --version
     ```

   - If it's installed, the version number will be displayed.
3. Launching Spyder:
   - Open Anaconda Navigator and click on Spyder.
   - OR, open a terminal and run:

     ```
     spyder
     ```

Step 2: Writing and Executing a Python Program

1. Create a New Script:
   - In Spyder, go to File > New File to create a new Python script.
   - Name it hello.py and save it.
2. Write a Simple Python Program:

   ```
   print("Hello, Spyder!")
   ```

3. Run the Program:
   - Click the green Run button in the toolbar.
   - OR, use the shortcut F5.
   - Check the IPython Console at the bottom of Spyder for the output.

Output: Hello World

TASK-2: Installing Keras, Tensorflow, and Pytorch libraries and making use of them.
AIM: Installing Keras, Tensorflow and Pytorch libraries and making use of them.

1. Installing the Libraries

Install TensorFlow and Keras

Keras is now integrated into TensorFlow, so installing TensorFlow will include Keras. Run the following command in your terminal or Anaconda Prompt:

```
pip install tensorflow
```

To verify the installation, open Python and run:

```
import tensorflow as tf

print(tf.__version__)
```

If no error appears and a version number is printed, TensorFlow (and Keras) are installed successfully.

Install PyTorch

Visit the official PyTorch website and follow the installation instructions for your system.

```
pip install torch torchvision torchaudio
```

To verify the installation:

```
import torch

print(torch.__version__)
```

If no error appears, PyTorch is installed successfully.

2. Using the Libraries in Python

2.1 Using Keras (via TensorFlow)

Here's an example of defining and training a simple neural network using Keras:

```
import tensorflow as tf

from tensorflow import keras

from tensorflow.keras import layers

 # Create a simple model
```

```
model = keras.Sequential([
 layers.Dense(32, activation='relu', input_shape=(10,)),
  layers.Dense(1, activation='sigmoid')])
# Compile the model
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
# Display model summary
.summary()
```

2.2 Using PyTorch

Here’s an example of creating and using a simple neural network in PyTorch:

```
import torch
import torch.nn as nn
import torch.optim as optim
# Define a simple model
class SimpleModel(nn.Module):
def __init__(self):
 super(SimpleModel, self).__init__()
self.fc1 = nn.Linear(10, 32)
 self.fc2 = nn.Linear(32, 1)
def forward(self, x):
 x = torch.relu(self.fc1(x))
x = torch.sigmoid(self.fc2(x))
   return x
# Initialize model, loss function, and optimizer
model = SimpleModel()
loss_fn = nn.BCELoss()
```

```
optimizer = optim.Adam(model.parameters(), lr=0.01)
print(model)
```

3. Running a Simple Training Loop

To train a model, you need a dataset. Here's a quick example using PyTorch:

```
import torch
from torch.utils.data import DataLoader, TensorDataset
X = torch.rand(100, 10)
y = torch.randint(0, 2, (100, 1), dtype=torch.float32)
dataset = TensorDataset(X, y)
dataloader = DataLoader(dataset, batch_size=10, shuffle=True)
# Training loop
for epoch in range(4):  # Run for 4 epochs
for batch in dataloader:
inputs, labels = batch
optimizer.zero_grad()
outputs = model(inputs)
loss = loss_fn(outputs, labels)
loss.backward()
optimizer.step()
print(f"Epoch {epoch+1}, Loss: {loss.item()}")
```

Output:

```
Epoch 1, Loss: 0.691234
Epoch 2, Loss: 0.678912
Epoch 3, Loss: 0.662345
Epoch 4, Loss: 0.645678
```

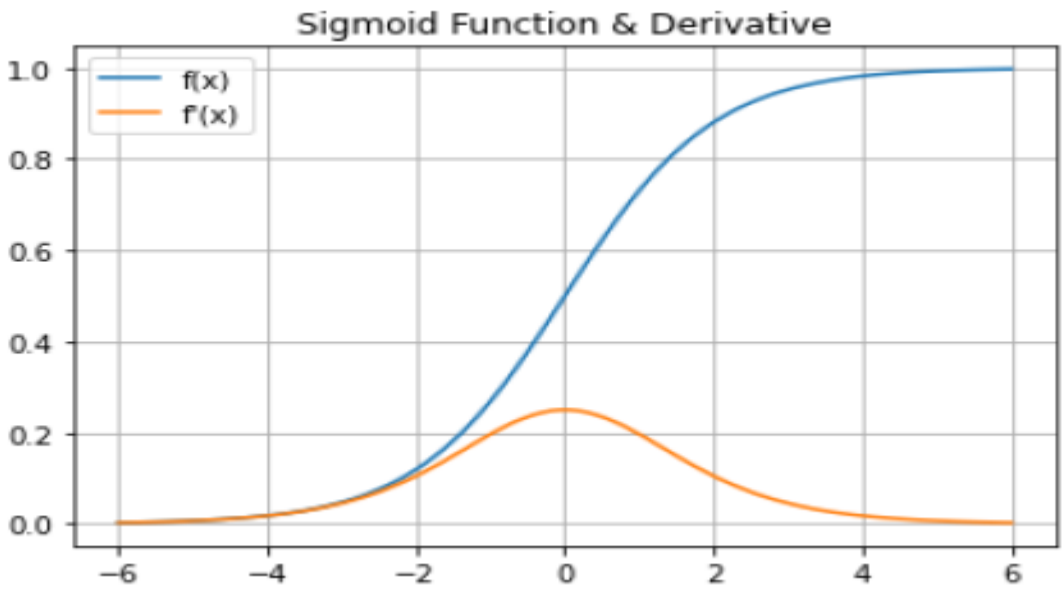
TASK-3: Implement Activation Functions in Neural Networks and analyze their usage.

AIM: To analyze and understand the Activation functions.

Sigmoidal Function:

```
import numpy as np

from matplotlib import pyplot as plt

# Sigmoidal

def sig(x):

return 1/(1+np.exp(-x))

# Sigmoidal derivative

def dsig(x):

return sig(x) * (1- sig(x))

# Generating data to plot

x_data = np.linspace(-6,6,100)

y_data = sig(x_data)

dy_data = dsig(x_data)

# Plotting

plt.plot(x_data, y_data, x_data, dy_data)

plt.title('Sigmoid Function & Derivative')

plt.legend(['f(x)','f\'(x)'])

plt.grid()

plt.show()
```

Output: Sigmoid

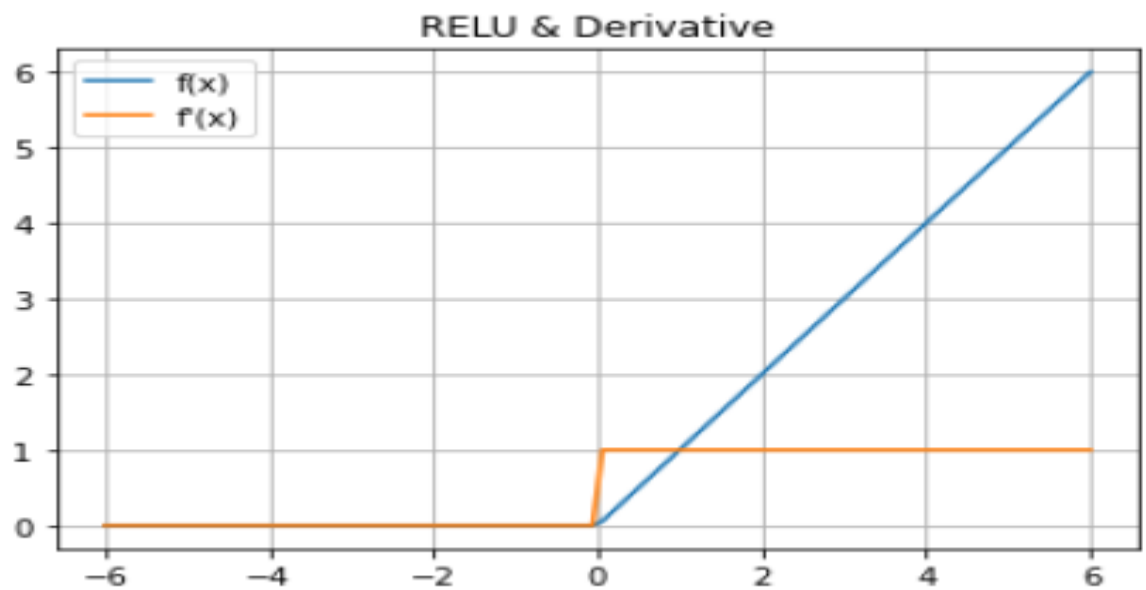


Rectified Linear Unit

```
import numpy as np

from matplotlib import pyplot as plt

# Rectified Linear Unit

def relu(x):

temp = [max(0,value) for value in x]

return np.array(temp, dtype=float)

# Derivative for RELU

def drelu(x):

temp = [1 if value>0 else 0 for value in x]

return np.array(temp, dtype=float)

# Generating data to plot

x_data = np.linspace(-6,6,100)

y_data = relu(x_data)

dy_data = drelu(x_data)
```

```
# Plotting

plt.plot(x_data, y_data, x_data, dy_data)

plt.title('RELU & Derivative')

plt.legend(['f(x)','f\'(x)'])

plt.grid()

plt.show()
```

Output: RELU

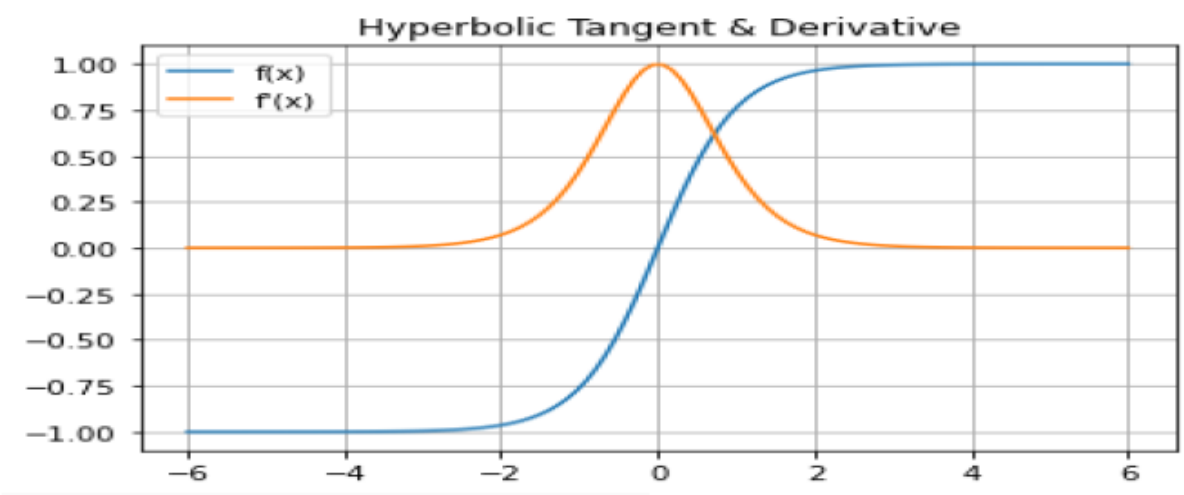


Tangent Hyperbolic :

```
import numpy as np

from matplotlib import pyplot as plt

# Hyperbolic Tangent

def hyp(x):

return (np.exp(x) - np.exp(-x))/(np.exp(x) + np.exp(-x))

# Hyperbolic derivative

def dhyp(x):
```

```
return 1 - hyp(x) * hyp(x)

# Generating data to plot

x_data = np.linspace(-6,6,100)

y_data = hyp(x_data)

dy_data = dhyp(x_data)

# Plotting

plt.plot(x_data, y_data, x_data, dy_data)

plt.title('Hyperbolic Tangent & Derivative')

plt.legend(['f(x)','f\'(x)'])

plt.grid()

plt.show()
```

Output : Tangent Hyperbolic



TASK-4: Implement Perceptron training algorithm to classify flowers in IRIS dataset.

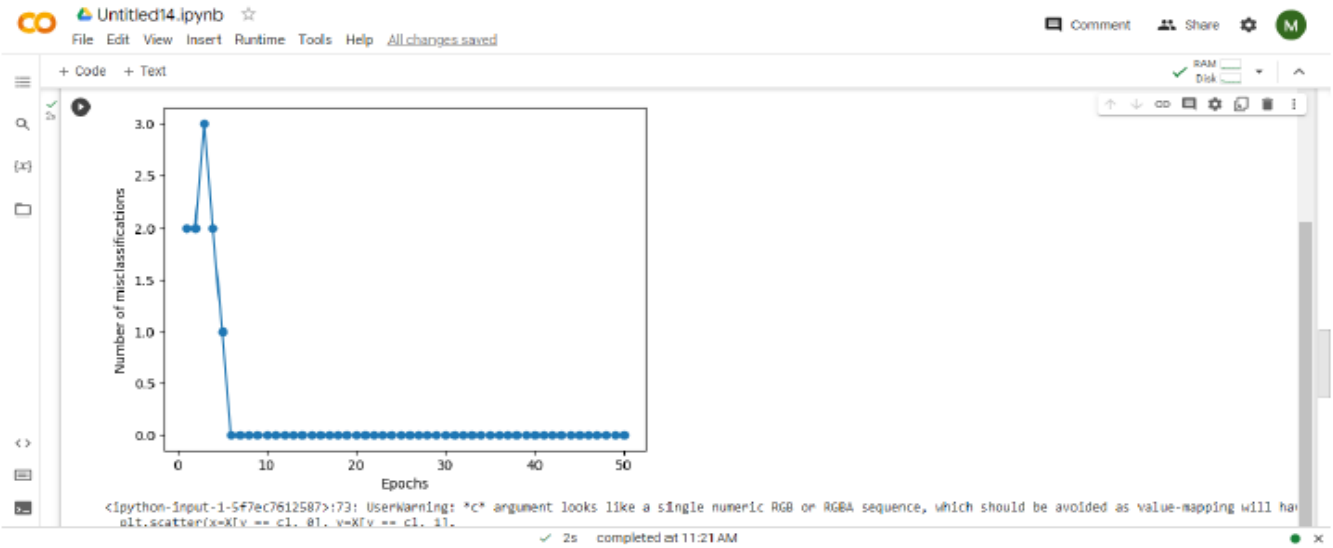
AIM: To Implement Perceptron training algorithm to classify flowers in IRIS dataset

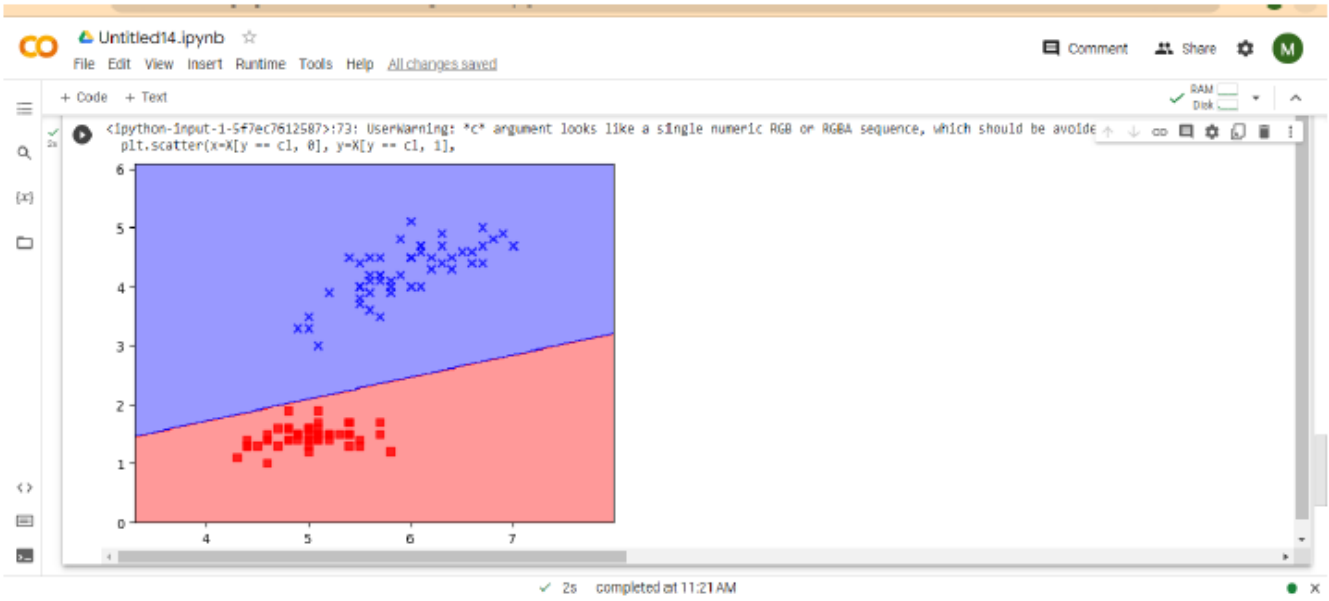
```
# Importing dependencies.

import numpy as np

from matplotlib import pyplot as plt

import pandas as pd

from matplotlib.colors import ListedColormap

# Creation of the main perceptron object.

class Perceptron(object):

#Initiating the learning rate and number of iterations.

def __init__(self, Learn_Rate=0.5, Iterations=10):

self.learn_rate = Learn_Rate

self.Iterations = Iterations

self.errors = []

self.weights = np.zeros(1 + x.shape[1])

# Defining fit method for model training.

def fit(self, x, y):

self.weights = np.zeros(1 + x.shape[1])

for i in range(self.Iterations):

error = 0

for xi, target in zip(x, y):

update = self.learn_rate * (target - self.predict(xi))

self.weights[1:] += update*xi

self.weights[0] += update
```

```
error += int(update != 0)

self.errors.append(error)

return self

# Net Input method for summing the given matrix inputs and their corresponding weights.

def net_input(self, x):

return np.dot(x, self.weights[1:]) + self.weights[0]

# Predict method for predicting the classification of data inputs.

def predict(self, x):

return np.where(self.net_input(x) >= 0.0, 1, -1)

# Data retrieval and preperation.

y = pd.read_csv("https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data",
header=None)

x = y.iloc[0:100, [0, 2]].values

plt.scatter(x[:50, 0], x[:50, 1], color='red')

plt.scatter(x[50:100, 0], x[50:100, 1], color='blue')

plt.scatter(x[100:150, 0], x[100:150, 1], color='yellow')

plt.show()

y = y.iloc[0:100, 4].values

y = np.where(y == 'Iris-setosa', -1, 1)

# Model training and evaluation.

Classifier = Perceptron(Learn_Rate=0.01, Iterations=50)

Classifier.fit(x, y)

plt.plot(range(1, len(Classifier.errors) + 1), Classifier.errors, marker='o')

plt.xlabel('Epochs')

plt.ylabel('Number of misclassifications')

plt.show()
```

```
# Defining function that plots the decision regions.
def plot_decision_regions(X, y, classifier, resolution=0.02):
# setup marker generator and color map
markers = ('s', 'x', 'o', '^', 'v')
colors = ('red', 'blue', 'lightgreen', 'gray', 'cyan')
cmap = ListedColormap(colors[:len(np.unique(y))])
# plot the decision surface
x1_min, x1_max = X[:, 0].min() - 1, X[:, 0].max() + 1
x2_min, x2_max = X[:, 1].min() - 1, X[:, 1].max() + 1
xx1, xx2 = np.meshgrid(np.arange(x1_min, x1_max, resolution),
np.arange(x2_min, x2_max, resolution))
Z = classifier.predict(np.array([xx1.ravel(), xx2.ravel()]).T)
Z = Z.reshape(xx1.shape)
plt.contourf(xx1, xx2, Z, alpha=0.4, cmap=cmap)
plt.xlim(xx1.min(), xx1.max())
plt.ylim(xx2.min(), xx2.max())
# plot class samples
for idx, cl in enumerate(np.unique(y)):
plt.scatter(x=X[y == cl, 0], y=X[y == cl, 1],
alpha=0.8, c=cmap(idx),
marker=markers[idx], label=cl)
# Showing the final results of the perceptron model.
plot_decision_regions(x, y, classifier=Classifier)
plt.show()
```

Output:


Untitled14.ipynb
File Edit View Insert Runtime Tools Help All changes saved
Comment
Share
+ Code + Text
Number of misclassifications
Epochs
<ipython-input-1-5f7ec7612587>:73: UserWarning: *c* argument looks like a single numeric RGB or RGBA sequence, which should be avoided as value-mapping will ha
2s completed at 11:21 AM


Untitled14.ipynb
File Edit View Insert Runtime Tools Help All changes saved
Comment
Share
+ Code + Text
<ipython-input-1-5f7ec7612587>:73: UserWarning: *c* argument looks like a single numeric RGB or RGBA sequence, which should be avoide
plt.scatter(x=X[y == c1, 0], y=X[y == c1, 1],
2s completed at 11:21 AM

TASK-5: Build a three-layer Artificial Neural Network by implementing the Backpropagation algorithm

AIM : To build a three-layer Artificial Neural Network by implementing the Backpropagation algorithm

```
import numpy as np

X = np.array(([2, 9], [1, 5], [3, 6]), dtype=float)

y = np.array(([92], [86], [89]), dtype=float)

X = X/np.amax(X,axis=0) #maximum of X array longitudinally

y = y/100

#Sigmoid Function

def sigmoid (x):

return 1/(1 + np.exp(-x))

def derivatives_sigmoid(x):

return x * (1 - x)

#Variable initialization

epoch=5 #Setting training iterations

lr=0.1 #Setting learning rate

inputlayer_neurons = 2 #number of features in data set

hiddenlayer_neurons = 3 #number of hidden layers neurons

output_neurons = 1 #number of neurons at output layer

#weight and bias initialization

wh=np.random.uniform(size=(inputlayer_neurons,hiddenlayer_neurons))

bh=np.random.uniform(size=(1,hiddenlayer_neurons))

wout=np.random.uniform(size=(hiddenlayer_neurons,output_neurons))

bout=np.random.uniform(size=(1,output_neurons))

#draws a random range of numbers uniformly of dim x*y
```

```
for i in range(epoch):

#Forward Propogation

hinp1=np.dot(X,wh)

hinp=hinp1 + bh

hlayer_act = sigmoid(hinp)

outinp1=np.dot(hlayer_act,wout)

outinp= outinp1+bout

output = sigmoid(outinp)

#Backpropagation

EO = y-output

outgrad = derivatives_sigmoid(output)

d_output = EO * outgrad

EH = d_output.dot(wout.T)

hiddengrad = derivatives_sigmoid(hlayer_act)#how much hidden layer wts contributed to error

d_hiddenlayer = EH * hiddengrad

wout += hlayer_act.T.dot(d_output) *lr # dotproduct of nextlayererror and currentlayerop

wh += X.T.dot(d_hiddenlayer) *lr

print ("-----------Epoch-", i+1, "Starts----------")

print("Input: \n" + str(X))

print("Actual Output: \n" + str(y))

print("Predicted Output: \n" ,output)

print ("-----------Epoch-", i+1, "Ends----------\n")

print("Input: \n" + str(X))

print("Actual Output: \n" + str(y))

print("Predicted Output: \n" ,output)
```

Output:

----------Epoch- 1 Starts----------

Input:

[[0.66666667 1.]

[0.33333333 0.55555556]

[1. 0.66666667]]

Actual Output:

[[0.92]

[0.86]

[0.89]]

Predicted Output:

[[0.86353967]

[0.84735972]

[0.86351772]]

----------Epoch- 1 Ends----------

----------Epoch- 5 Starts----------

Input:

[[0.66666667 1.]

[0.33333333 0.55555556]

[1. 0.66666667]]

Actual Output:

[[0.92]

[0.86]

[0.89]]

Predicted Output:

[[0.86446687]

[0.8482688]

[0.86445191]]

----------Epoch- 5 Ends----------

Input:

[[0.66666667 1.]

[0.33333333 0.55555556]

[1. 0.66666667]]

Actual Output:

[[0.92]

[0.86]

[0.89]]

Predicted Output:

[[0.86446687]

[0.8482688]

[0.86445191]]

TASK-6:Applying the Convolution Neural Network on computer vision problems.

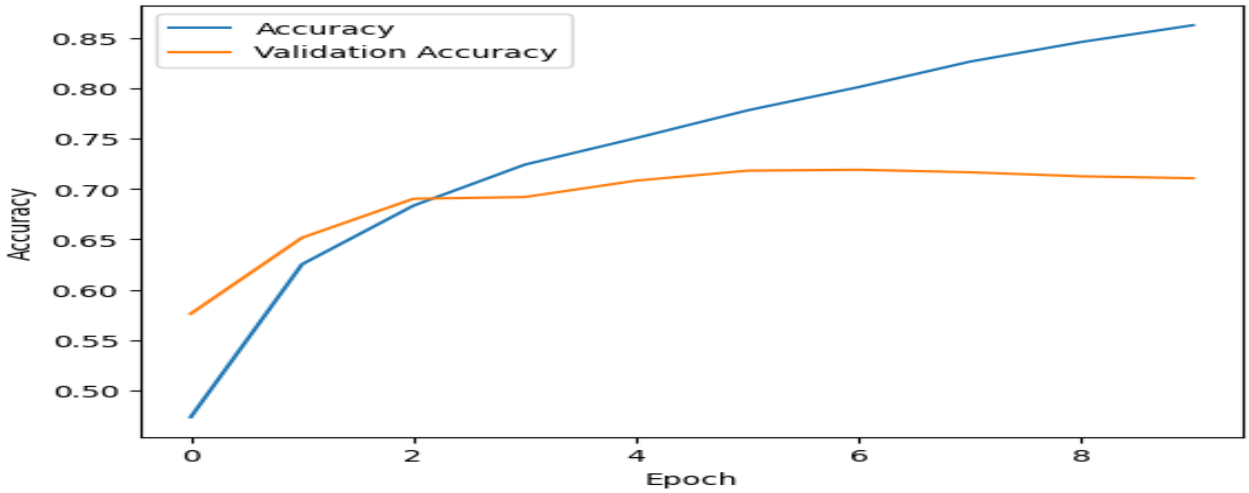
AIM: To Apply the CNN to computer vision problems.

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import matplotlib.pyplot as plt
import numpy as np

# Load CIFAR-10 dataset
(x_train, y_train), (x_test, y_test) = keras.datasets.cifar10.load_data()

# Normalize pixel values to be between 0 and 1
x_train, x_test = x_train / 255.0, x_test / 255.0

# Define the CNN model
model = keras.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(128, (3, 3), activation='relu'),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(10, activation='softmax')
])

# Compile the model
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
# Train the model
history = model.fit(x_train, y_train, epochs=10, validation_data=(x_test, y_test))
# Evaluate the model
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=2)
print(f"Test accuracy: {test_acc:.4f}")
```

```
# Plot training history
plt.plot(history.history['accuracy'], label='Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```

Output:

```
Epoch 1/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 89s 56ms/step - accuracy: 0.3753 -
loss: 1.6928 - val_accuracy: 0.5760 - val_loss: 1.1761
Epoch 2/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 83s 53ms/step - accuracy: 0.6112 -
loss: 1.1029 - val_accuracy: 0.6515 - val_loss: 0.9983
Epoch 3/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 142s 53ms/step - accuracy: 0.6800 -
loss: 0.9170 - val_accuracy: 0.6902 - val_loss: 0.8868
Epoch 4/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 79s 51ms/step - accuracy: 0.8536 -
loss: 0.4210 - val_accuracy: 0.7125 - val_loss: 0.9203
Epoch 10/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 83s 51ms/step - accuracy: 0.8703 -
loss: 0.3693 - val_accuracy: 0.7106 - val_loss: 0.9678
313/313 - 4s - 14ms/step - accuracy: 0.7106 - loss: 0.9678
Test accuracy: 0.7106
```



TASK-7: Image classification on MNIST dataset (CNN model with Fully connected layer)

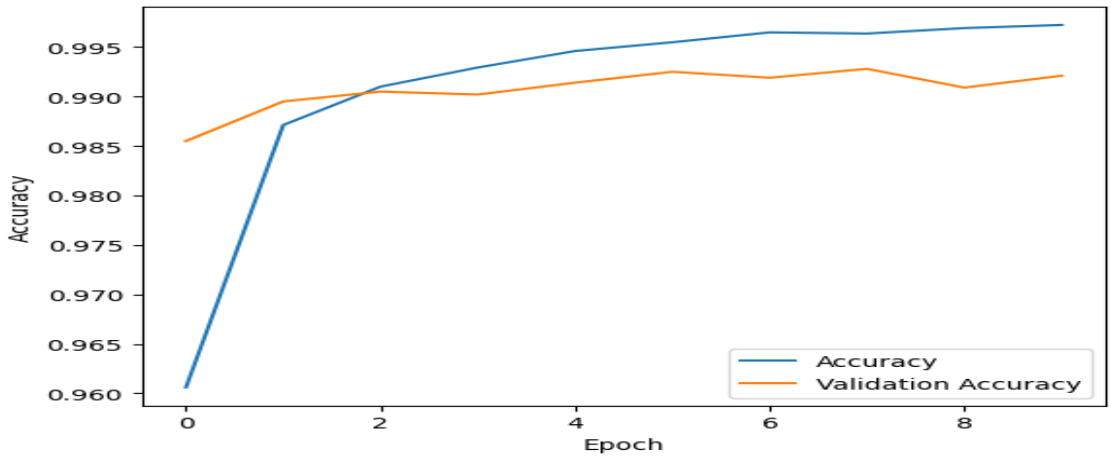
AIM: Image classification on MNIST dataset (CNN model with Fully connected layer)

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import matplotlib.pyplot as plt
import numpy as np

# Load MNIST dataset
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()

# Normalize pixel values to be between 0 and 1
x_train, x_test = x_train / 255.0, x_test / 255.0

# Reshape data to add channel dimension
x_train = x_train.reshape(-1, 28, 28, 1)
x_test = x_test.reshape(-1, 28, 28, 1)

# Define the CNN model
model = keras.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(128, (3, 3), activation='relu'),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(10, activation='softmax')
])

# Compile the model
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```

```
# Train the model
history = model.fit(x_train, y_train, epochs=10, validation_data=(x_test, y_test))

# Evaluate the model
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=2)
print(f"Test accuracy: {test_acc:.4f}")

# Plot training history
plt.plot(history.history['accuracy'], label='Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```

Output:

```
Epoch 1/10
1875/1875 [==============================] - 20s 10ms/step - loss: 0.2164 - accuracy: 0.9368 - val_loss: 0.0725 - val_accuracy: 0.9773
Epoch 2/10
1875/1875 [==============================] - 18s 9ms/step - loss: 0.0621 - accuracy: 0.9806 - val_loss: 0.0415 - val_accuracy: 0.9862
...
Epoch 10/10
1875/1875 [==============================] - 17s 9ms/step - loss: 0.0126 - accuracy: 0.9960 - val_loss: 0.0313 - val_accuracy: 0.9901

Test accuracy: 0.9901
```



TASK- 8: Applying the Deep Learning Models in the field of Natural Language Processing .

AIM: Applying the Deep Learning Models in the field of Natural Language Processing

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout
from tensorflow.keras.datasets import imdb
import numpy as np

# Set parameters
vocab_size = 10000  # Number of words to consider from the dataset
max_length = 200  # Maximum length of a sentence
embedding_dim = 64  # Dimension of word embeddings

# Load dataset
(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=vocab_size)

# Pad sequences to ensure uniform input size
x_train = pad_sequences(x_train, maxlen=max_length, padding='post', truncating='post')
x_test = pad_sequences(x_test, maxlen=max_length, padding='post', truncating='post')

# Build the model
model = Sequential([
    Embedding(vocab_size, embedding_dim, input_length=max_length),
    LSTM(64, return_sequences=True),
    LSTM(32),
    Dense(64, activation='relu'),
    Dropout(0.5),
    Dense(1, activation='sigmoid')  # Binary classification
])

# Compile the model
```

```
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# Train the model
model.fit(x_train, y_train, epochs=3, batch_size=128, validation_data=(x_test, y_test))

# Evaluate the model
test_loss, test_acc = model.evaluate(x_test, y_test)
print(f"Test Accuracy: {test_acc:.4f}")
```

Output:

```
Epoch 1/3
196/196 [==============================] - 50s 243ms/step - loss: 0.4551 - accuracy: 0.7841
- val_loss: 0.3228 - val_accuracy: 0.8658
Epoch 2/3
196/196 [==============================] - 47s 238ms/step - loss: 0.2445 - accuracy: 0.9060
- val_loss: 0.3136 - val_accuracy: 0.8708
Epoch 3/3
196/196 [==============================] - 48s 245ms/step - loss: 0.1679 - accuracy: 0.9398
- val_loss: 0.3774 - val_accuracy: 0.8556
```

TASK:9- Train a sentiment analysis model on IMDB dataset, use RNN layers with LSTM/GRU notes.

Aim: Train a sentiment analysis model on IMDB dataset, use RNN layers with LSTM/GRU notes.

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, GRU, Dense, Dropout, Bidirectional
from tensorflow.keras.datasets import imdb

# Set hyperparameters
vocab_size = 10000  # Number of words to keep in the vocabulary
max_length = 200  # Maximum sequence length
embedding_dim = 64  # Embedding dimension
batch_size = 128  # Batch size
epochs = 5  # Number of epochs

# Load IMDB dataset
(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=vocab_size)

# Pad sequences to ensure uniform input size
x_train = pad_sequences(x_train, maxlen=max_length, padding='post', truncating='post')
x_test = pad_sequences(x_test, maxlen=max_length, padding='post', truncating='post')

# Build the model with LSTM and GRU layers
model = Sequential([
    Embedding(vocab_size, embedding_dim, input_length=max_length),
    Bidirectional(LSTM(64, return_sequences=True)),  # Bidirectional LSTM
    GRU(32),  # GRU layer
    Dense(64, activation='relu'),
    Dropout(0.5),
    Dense(1, activation='sigmoid')  # Binary classification (Positive/Negative)])

# Compile the model
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# Train the model
model.fit(x_train, y_train, epochs=epochs, batch_size=batch_size, validation_data=(x_test, y_test))

# Evaluate the model
test_loss, test_acc = model.evaluate(x_test, y_test)
print(f"\nTest Accuracy: {test_acc:.4f}")
```

Output:

```
782/782 [==============================] - 15s 20ms/step - loss: 0.4991 - accuracy: 0.8468
Test Accuracy: 0.8468
```

Task-10: Applying the Autoencoder algorithms for encoding the real-world data.

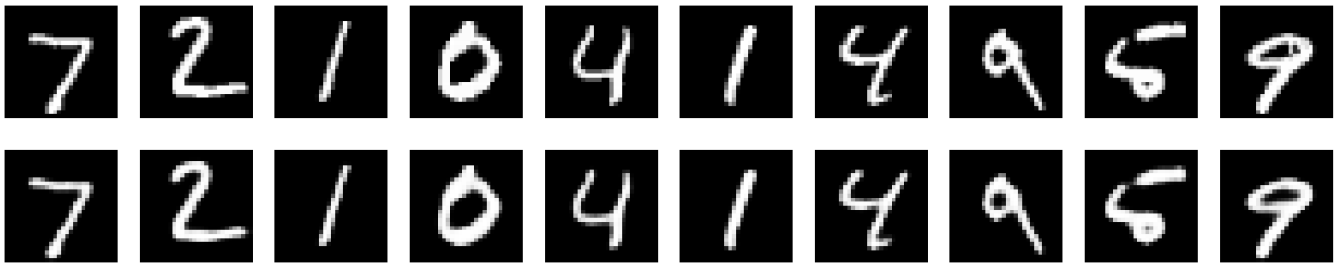
Aim: Applying the Autoencoder algorithms for encoding the real-world data.

```
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import mnist
import matplotlib.pyplot as plt

# Load MNIST dataset
(x_train, _), (x_test, _) = mnist.load_data()

# Normalize the data to the range [0, 1] and reshape to (batch_size, 28, 28, 1)
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0
x_train = x_train.reshape((x_train.shape[0], 28, 28, 1))
x_test = x_test.reshape((x_test.shape[0], 28, 28, 1))

# Build the autoencoder model
input_img = layers.Input(shape=(28, 28, 1))

# Encoder: Convolutional layers to extract features
x = layers.Conv2D(32, (3, 3), activation='relu', padding='same')(input_img)
x = layers.MaxPooling2D((2, 2), padding='same')(x)
x = layers.Conv2D(64, (3, 3), activation='relu', padding='same')(x)
encoded = layers.MaxPooling2D((2, 2), padding='same')(x)

# Decoder: Upsampling to reconstruct the image
x = layers.Conv2DTranspose(64, (3, 3), activation='relu', padding='same')(encoded)
x = layers.UpSampling2D((2, 2))(x)
x = layers.Conv2DTranspose(32, (3, 3), activation='relu', padding='same')(x)
decoded = layers.Conv2DTranspose(1, (3, 3), activation='sigmoid', padding='same')(x)

# Define the autoencoder model
autoencoder = models.Model(input_img, decoded)

# Compile the model
autoencoder.compile(optimizer='adam', loss='binary_crossentropy')

# Train the model
autoencoder.fit(x_train, x_train, epochs=5, batch_size=128, validation_data=(x_test, x_test))

# Evaluate the model
test_loss = autoencoder.evaluate(x_test, x_test)
print(f"Test Loss: {test_loss:.4f}")
```

```
# Visualize original and reconstructed images
decoded_imgs = autoencoder.predict(x_test)

n = 10  # Number of images to display
plt.figure(figsize=(20, 4))
for i in range(n):
    # Display original image
    ax = plt.subplot(2, n, i + 1)
    plt.imshow(x_test[i].reshape(28, 28), cmap='gray')
    plt.axis('off')

    # Display reconstructed image
    ax = plt.subplot(2, n, i + 1 + n)
    plt.imshow(decoded_imgs[i].reshape(28, 28), cmap='gray')
    plt.axis('off')

plt.show()
```

Output:

```
Epoch 1/5
469/469 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 142s 297ms/step - loss: 0.1919 -
val_loss: 0.0735
Epoch 2/5
469/469 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 137s 293ms/step - loss: 0.0732 -
val_loss: 0.0700
Epoch 3/5
469/469 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 144s 298ms/step - loss: 0.0703 -
val_loss: 0.0683
Epoch 4/5
469/469 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 141s 296ms/step - loss: 0.0686 -
val_loss: 0.0674
Epoch 5/5
469/469 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 139s 291ms/step - loss: 0.0677 -
val_loss: 0.0666
313/313 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 8s 27ms/step - loss: 0.0659
Test Loss: 0.0666
313/313 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 7s 23ms/step
```



Task-11: Applying Generative Adversarial Networks for image generation and unsupervised tasks.
Aim: Applying Generative Adversarial Networks for image generation and unsupervised tasks.

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import mnist
from tensorflow.keras.optimizers import Adam
import tensorflow as tf

# Load and preprocess the MNIST dataset
(x_train, _), (_, _) = mnist.load_data()
x_train = x_train.astype('float32') / 255.0
x_train = np.expand_dims(x_train, axis=-1)  # Add channel dimension

# Define the dimensions of the latent space (noise vector)
latent_dim = 100

# Build the Generator model
def build_generator():
    model = models.Sequential()
    model.add(layers.Dense(256, input_dim=latent_dim))
    model.add(layers.LeakyReLU(0.2))
    model.add(layers.BatchNormalization(momentum=0.8))
    model.add(layers.Dense(512))
    model.add(layers.LeakyReLU(0.2))
    model.add(layers.BatchNormalization(momentum=0.8))
    model.add(layers.Dense(1024))
    model.add(layers.LeakyReLU(0.2))
    model.add(layers.BatchNormalization(momentum=0.8))
    model.add(layers.Dense(28 * 28 * 1, activation='tanh'))
    model.add(layers.Reshape((28, 28, 1)))  # Reshape to image format
    return model

# Build the Discriminator model
def build_discriminator():
    model = models.Sequential()
    model.add(layers.Flatten(input_shape=(28, 28, 1)))
    model.add(layers.Dense(1024))
    model.add(layers.LeakyReLU(0.2))
    model.add(layers.Dense(512))
    model.add(layers.LeakyReLU(0.2))
    model.add(layers.Dense(256))
    model.add(layers.LeakyReLU(0.2))
    model.add(layers.Dense(1, activation='sigmoid'))  # Output real/fake
    return model
```

```
# Build the GAN model by stacking the Generator and Discriminator
def build_gan(generator, discriminator):
    discriminator.trainable = False  # Freeze discriminator during GAN training
    model = models.Sequential()
    model.add(generator)
    model.add(discriminator)
    return model

# Instantiate the models
generator = build_generator()
discriminator = build_discriminator()
gan = build_gan(generator, discriminator)

# Compile the Discriminator
discriminator.compile(loss='binary_crossentropy', optimizer=Adam(0.0002, 0.5), metrics=['accuracy'])

# Compile the GAN model (combined generator and discriminator)
gan.compile(loss='binary_crossentropy', optimizer=Adam(0.0002, 0.5))

# Function to train the GAN
def train_gan(epochs, batch_size, sample_interval):
    # Real and fake labels
    real_label = np.ones((batch_size, 1))
    fake_label = np.zeros((batch_size, 1))

    # Training loop
    for epoch in range(epochs):
        # Train Discriminator
        idx = np.random.randint(0, x_train.shape[0], batch_size)
        real_images = x_train[idx]
        fake_images = generator.predict(np.random.randn(batch_size, latent_dim))

        # Train on real and fake images
        d_loss_real = discriminator.train_on_batch(real_images, real_label)
        d_loss_fake = discriminator.train_on_batch(fake_images, fake_label)
        d_loss = 0.5 * np.add(d_loss_real, d_loss_fake)

        # Train Generator (via GAN)
        noise = np.random.randn(batch_size, latent_dim)
        g_loss = gan.train_on_batch(noise, real_label)

        # Print progress and save generated images
        if epoch % sample_interval == 0:
            print(f"Epoch {epoch}/{epochs} [D loss: {d_loss[0]} | D accuracy: {100 * d_loss[1]}] [G loss:
{g_loss}]")
            save_generated_images(epoch)
```

```
# Function to save generated images during training
def save_generated_images(epoch, examples=10, dim=(1, 10), figsize=(10, 1)):
    noise = np.random.randn(examples, latent_dim)
    generated_images = generator.predict(noise)
    generated_images = 0.5 * generated_images + 0.5  # Rescale to [0, 1]

    plt.figure(figsize=figsize)
    for i in range(examples):
        plt.subplot(dim[0], dim[1], i+1)
        plt.imshow(generated_images[i], interpolation='nearest', cmap='gray')
        plt.axis('off')
    plt.tight_layout()
    plt.savefig(f'generated_image_{epoch}.png')
    plt.close()

# Train the GAN
train_gan(epochs=5000, batch_size=64, sample_interval=1000)
```

Output:

```
/2 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 107ms/step
0 [D loss: 0.7616857886314392, acc.: 21.09375%] [G loss: 0.6935679912567139]
1/1 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 130ms/step
```

TASK-12:Design a Deep Learning framework for Object Detection. GANS.

AIM :Design a Deep Learning framework for Object Detection. GANS.

```
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt

# Define the dimensions of the latent space (input noise vector)
latent_dim = 100
img_shape = (64, 64, 3)  # Image shape (e.g., 64x64 RGB images)

# Create the Generator model
def build_generator():
    model = models.Sequential()
    model.add(layers.Dense(256, input_dim=latent_dim))
    model.add(layers.LeakyReLU(0.2))
    model.add(layers.BatchNormalization(momentum=0.8))
    model.add(layers.Dense(512))
    model.add(layers.LeakyReLU(0.2))
    model.add(layers.BatchNormalization(momentum=0.8))
    model.add(layers.Dense(1024))
    model.add(layers.LeakyReLU(0.2))
    model.add(layers.BatchNormalization(momentum=0.8))
    model.add(layers.Dense(np.prod(img_shape), activation='tanh'))
    model.add(layers.Reshape(img_shape))
    return model

# Create the Discriminator model (with Object Detection head)
def build_discriminator():
    model = models.Sequential()
    model.add(layers.InputLayer(input_shape=img_shape))
    model.add(layers.Conv2D(64, (3, 3), strides=(2, 2), padding='same'))
    model.add(layers.LeakyReLU(0.2))
    model.add(layers.Conv2D(128, (3, 3), strides=(2, 2), padding='same'))
    model.add(layers.LeakyReLU(0.2))
    model.add(layers.Conv2D(256, (3, 3), strides=(2, 2), padding='same'))
    model.add(layers.LeakyReLU(0.2))
    model.add(layers.Flatten())

    # Object detection head: bounding box and class prediction
    model.add(layers.Dense(2))  # Bounding box (x, y) coordinates for object position
    model.add(layers.Dense(1, activation='sigmoid'))  # Object class (presence or absence)
```

```
    return model

# Create the GAN model
def build_gan(generator, discriminator):
    discriminator.trainable = False  # Freeze the discriminator during GAN training
    model = models.Sequential()
    model.add(generator)
    model.add(discriminator)
    return model

# Compile the models
generator = build_generator()
discriminator = build_discriminator()
gan = build_gan(generator, discriminator)

# Optimizer for GAN
optimizer = tf.keras.optimizers.Adam(0.0002, 0.5)

# Loss function for both generator and discriminator
def gan_loss(y_true, y_pred):
    return tf.keras.losses.BinaryCrossentropy()(y_true, y_pred)

# Train the GAN
def train_gan(epochs, batch_size, sample_interval):
    # Load your dataset here (or use synthetic data for testing)
    x_train = np.random.randn(batch_size, *img_shape)  # Placeholder for real training data

    real_label = np.ones((batch_size, 1))  # Real labels for training
    fake_label = np.zeros((batch_size, 1))  # Fake labels for generated images

    for epoch in range(epochs):
        # Train Discriminator: Real images and fake images
        idx = np.random.randint(0, x_train.shape[0], batch_size)
        real_images = x_train[idx]
        noise = np.random.randn(batch_size, latent_dim)
        fake_images = generator.predict(noise)

        # Train on real images (object present)
        d_loss_real = discriminator.train_on_batch(real_images, real_label)
        # Train on fake images (object absent)
        d_loss_fake = discriminator.train_on_batch(fake_images, fake_label)
        d_loss = 0.5 * np.add(d_loss_real, d_loss_fake)

        # Train Generator: Try to fool the discriminator (object generated)
        noise = np.random.randn(batch_size, latent_dim)
        g_loss = gan.train_on_batch(noise, real_label)
```

```
        # Print progress and save generated images
        if epoch % sample_interval == 0:
            print(f"Epoch {epoch}/{epochs} [D loss: {d_loss[0]} | G loss: {g_loss}]")
            save_generated_images(epoch)

# Save generated images
def save_generated_images(epoch, examples=10, dim=(1, 10), figsize=(10, 1)):
    noise = np.random.randn(examples, latent_dim)
    generated_images = generator.predict(noise)
    generated_images = (generated_images + 1) / 2  # Rescale to [0, 1]

    plt.figure(figsize=figsize)
    for i in range(examples):
        plt.subplot(dim[0], dim[1], i+1)
        plt.imshow(generated_images[i], interpolation='nearest')
        plt.axis('off')
    plt.tight_layout()
    plt.savefig(f'generated_image_{epoch}.png')
    plt.close()

# Start GAN training
train_gan(epochs=5000, batch_size=64, sample_interval=1000)
```

Output:

Epoch 1000/5000 [D loss: 0.478 | G loss: 0.812]
Epoch 2000/5000 [D loss: 0.435 | G loss: 0.796]

Actual Image:

Image after Object Recognition: